GitHub Project Explorer

Project Report | React web application using the GitHub REST API | November 2026

1. Introduction

Open-source software is the foundation of modern development, but finding good projects among millions of repositories is difficult. Developers usually have to browse GitHub page by page, compare projects manually and keep their own notes elsewhere. **GitHub Project Explorer** is a single-page web application that solves this by bringing discovery, comparison, bookmarking and analytics together in one clean, responsive interface, powered by live data from GitHub.

2. Abstract

The project is a front-end application built with React and Tailwind CSS that consumes the public GitHub REST API. It provides five modules: a **Dashboard** with summary cards and Chart.js charts (language distribution, top repositories by stars and forks); an **Explorer** with debounced keyword search, language filtering, sorting and pagination; a **Trending** view of fast-growing new repositories; a **Comparison** tool that places up to four repositories side by side and highlights the best values; and a **Bookmarks** manager where users save repositories and attach personal notes. The application uses a layered architecture (service layer, custom hook, React Context, UI components), handles API rate limits and errors gracefully, shows a live request counter, and stores user data in the browser's localStorage, so no backend or database is required.

3. Tools Used

Tool / Technology	Purpose in the project
React 18	Component-based user interface and state management (hooks, Context API)
React Router 6	Client-side routing between Dashboard, Explorer, Trending, Compare and Bookmarks
Tailwind CSS 3	Utility-first styling and responsive layout (mobile drawer menu, cards, tables)
Chart.js + react-chartjs-2	Doughnut and bar charts for the Dashboard and the comparison view
Axios	HTTP client with interceptors for rate-limit tracking and friendly error messages
GitHub REST API	Source of repository data (search, repository details, rate-limit endpoints)
Create React App (Node.js, npm)	Project tooling, development server, testing and production build
localStorage, VS Code, Git	Persistence of bookmarks, notes and comparison list; development environment; version control

4. Steps Involved in Building the Project

1. **Requirement analysis:** defined the goal, the five modules and the data needed from the GitHub API.
2. **Architecture and folder structure:** designed a scalable layout separating components (by feature), context, hooks, services and utilities.
3. **Project setup:** created the React project, configured Tailwind CSS, added environment variables for the optional GitHub access token and a .gitignore.
4. **Service layer:** built githubService.js to call the API, normalize repository data, track rate-limit headers and convert HTTP errors into readable messages.
5. **Custom hook and state:** created useGitHubApi (loading, error and data handling that ignores outdated responses) and AppContext (bookmarks, notes, comparison list, saved to localStorage).
6. **Shared components:** developed the Navbar, mobile Sidebar, Footer, Loading spinner, Error alert, Pagination and a reusable repository card.
7. **Feature modules:** implemented the Explorer (search, filters, sorting, pagination), Trending, Comparison, Bookmarks with a notes dialog, and the Dashboard with charts.
8. **Integration:** connected all pages through React Router in App.jsx and wired global state and toast notifications.

9. **Testing and documentation:** wrote unit tests for formatting utilities, verified the service logic with simulated API responses, ran the app locally with npm start, and prepared a README with setup instructions.

5. Conclusion

GitHub Project Explorer successfully combines search, trending discovery, side-by-side comparison, personal bookmarking and visual analytics in one lightweight application that runs entirely in the browser. The project demonstrates practical skills in React development, REST API integration, state management, data visualization, error handling and clean, modular architecture. Possible future improvements include user accounts with cloud-synced bookmarks, a small backend proxy to keep the access token private, dark mode, and deeper repository insights such as commit activity and contributor statistics.